Elasticsearch Guide:

8.8 (current)

What is Elasticsearch?

What's new in 8.8

Set up Elasticsearch

Upgrade Elasticsearch

Index modules

Analysis

Index Shard Allocation

Index blocks

Mapper

Merge

Similarity module

Slow Log

Store

Translog

History retention

Index Sorting

Indexing pressure

Mapping

Text analysis

Index templates

Data streams

Ingest pipelines

Aliases

Search your data

Query DSL

Aggregations

Index modules

Index Modules are modules created per index and control all aspects related to an index.

Index Settings

Index level settings can be set per-index. Settings may be:

static
: They can only be set at index creation time or on a closed index.

dynamic
: They can be changed on a live index using the update-index-settings API.

> **WARNING** Changing static or dynamic index settings on a closed index could result in incorrect settings that are impossible to rectify without deleting and recreating the index.

Static index settings

Below is a list of all *static* index settings that are not associated with any specific index module:

`index.number_of_shards`
: The number of primary shards that an index should have. Defaults to `1`. This setting can only be set at index creation time. It cannot be changed on a closed index.

> **NOTE** The number of shards are limited to `1024` per index. This limitation is a safety limit to prevent accidental creation of indices that can destabilize a cluster due to resource allocation. The limit can be modified by specifying `export ES_JAVA_OPTS="-Des.index.max_number_of_shards=128"` system property on every node that is part of the cluster.

`index.number_of_routing_shards`
: Integer value used with `index.number_of_shards` to route documents to a primary shard. See `_routing` field.

Elasticsearch uses this value when splitting an index. For example, a 5 shard index with `number_of_routing_shards` set to `30` (`5 x 2 x 3`) could be split by a factor of `2` or `3`. In other words, it could be split as follows:

Geospatial analysis
EQL
SQL
Scripting
Data management
Autoscaling
Monitor a cluster
Roll up or transform your data
Set up a cluster for high availability
Snapshot and restore
Secure the Elastic Stack
Watcher
Command line tools
How to
Troubleshooting
REST APIs
Migration guide
Release notes
Dependencies and versions

- `5` → `10` → `30` (split by 2, then by 3)
- `5` → `15` → `30` (split by 3, then by 2)
- `5` → `30` (split by 6)

This setting's default value depends on the number of primary shards in the index. The default is designed to allow you to split by factors of 2 up to a maximum of 1024 shards.

> **NOTE** In Elasticsearch 7.0.0 and later versions, this setting affects how documents are distributed across shards. When reindexing an older index with custom routing, you must explicitly set `index.number_of_routing_shards` to maintain the same document distribution. See the related breaking change.

`index.codec`
The `default` value compresses stored data with LZ4 compression, but this can be set to `best_compression` which uses DEFLATE for a higher compression ratio, at the expense of slower stored fields performance. If you are updating the compression type, the new one will be applied after segments are merged. Segment merging can be forced using force merge.

`index.routing_partition_size`
The number of shards a custom routing value can go to. Defaults to 1 and can only be set at index creation time. This value must be less than the `index.number_of_shards` unless the `index.number_of_shards` value is also 1. See Routing to an index partition for more details about how this setting is used.

`index.soft_deletes.enabled`
[~~7.6.0~~] Indicates whether soft deletes are enabled on the index. Soft deletes can only be configured at index creation and only on indices created on or after Elasticsearch 6.5.0. Defaults to `true`.

`index.soft_deletes.retention_lease.period`
The maximum period to retain a shard history retention lease before it is considered expired. Shard history retention leases ensure that soft deletes are retained during merges on the Lucene index. If a soft delete is merged away before it can be replicated to a follower the following process will fail due to incomplete history on the leader. Defaults to `12h`.

`index.load_fixed_bitset_filters_eagerly`
Indicates whether cached filters are pre-loaded for nested queries. Possible values are `true` (default) and `false`.

`index.shard.check_on_startup`

> **WARNING** Expert users only. This setting enables some very expensive processing at shard startup and is only ever useful while diagnosing a problem in your cluster. If you do use it, you should do so only temporarily and remove it once it is no longer needed.

Elasticsearch automatically performs integrity checks on the contents of shards at various points during their lifecycle. For instance, it verifies the checksum of every file transferred when recovering a replica or taking a snapshot. It also verifies the integrity of many important files when opening a shard, which happens when starting up a node and when finishing a shard recovery or relocation. You can therefore manually verify the integrity of a whole shard while it is running by taking a snapshot of it into a fresh repository or by recovering it onto a fresh node.

This setting determines whether Elasticsearch performs additional integrity checks while opening a shard. If these checks detect corruption then they will prevent the shard from being opened. It accepts the following values:

`false`
Don't perform additional checks for corruption when opening a shard. This is the default and recommended behaviour.

`checksum`
: Verify that the checksum of every file in the shard matches its contents. This will detect cases where the data read from disk differ from the data that Elasticsearch originally wrote, for instance due to undetected disk corruption or other hardware failures. These checks require reading the entire shard from disk which takes substantial time and IO bandwidth and may affect cluster performance by evicting important data from your filesystem cache.

`true`
: Performs the same checks as `checksum` and also checks for logical inconsistencies in the shard, which could for instance be caused by the data being corrupted while it was being written due to faulty RAM or other hardware failures. These checks require reading the entire shard from disk which takes substantial time and IO bandwidth, and then performing various checks on the contents of the shard which take substantial time, CPU and memory.

Dynamic index settings

edit

Below is a list of all *dynamic* index settings that are not associated with any specific index module:

`index.number_of_replicas`
: The number of replicas each primary shard has. Defaults to 1.

```
WARNING: Configuring it to 0 may lead to temporary availability loss
during node restarts or permanent data loss in case of data corruption.
```

`index.auto_expand_replicas`
: Auto-expand the number of replicas based on the number of data nodes in the cluster. Set to a dash delimited lower and upper bound (e.g. `0-5`) or use `all` for the upper bound (e.g. `0-all`). Defaults to `false` (i.e. disabled). Note that the auto-expanded number of replicas only takes allocation filtering rules into account, but ignores other allocation rules such as total shards per node, and this can lead to the cluster health becoming `YELLOW` if the applicable rules prevent all the replicas from being allocated.

  If the upper bound is `all` then shard allocation awareness and `cluster.routing.allocation.same_shard.host` are ignored for this index.

`index.search.idle.after`
: How long a shard can not receive a search or get request until it's considered search idle. (default is `30s`)

`index.refresh_interval`
: How often to perform a refresh operation, which makes recent changes to the index visible to search. Defaults to `1s`. Can be set to `-1` to disable refresh. If this setting is not explicitly set, shards that haven't seen search traffic for at least `index.search.idle.after` seconds will not receive background refreshes until they receive a search request. Searches that hit an idle shard where a refresh is pending will wait for the next background refresh (within `1s`). This behavior aims to automatically optimize bulk indexing in the default case when no searches are performed. In order to opt out of this behavior an explicit value of `1s` should set as the refresh interval.

`index.max_result_window`
: The maximum value of `from + size` for searches to this index. Defaults to `10000`. Search requests take heap memory and time proportional to `from + size` and this limits that memory. See Scroll or Search After for a more efficient alternative to raising this.

`index.max_inner_result_window`
: The maximum value of `from + size` for inner hits definition and top hits aggregations to this index. Defaults to `100`. Inner hits and top hits aggregation take heap memory and time proportional to `from + size` and this limits that memory.

`index.max_rescore_window`
: The maximum value of `window_size` for `rescore` requests in searches of this index. Defaults to `index.max_result_window` which defaults to `10000`. Search requests take heap memory and time proportional to `max(window_size, from + size)` and this limits that memory.

`index.max_docvalue_fields_search`
: The maximum number of `docvalue_fields` that are allowed in a query. Defaults to `100`. Doc-value fields are costly since they might incur a per-field per-document seek.

`index.max_script_fields`
: The maximum number of `script_fields` that are allowed in a query. Defaults to `32`.

`index.max_ngram_diff`
: The maximum allowed difference between min_gram and max_gram for NGramTokenizer and NGramTokenFilter. Defaults to `1`.

`index.max_shingle_diff`
: The maximum allowed difference between max_shingle_size and min_shingle_size for the `shingle` token filter. Defaults to `3`.

`index.max_refresh_listeners`
: Maximum number of refresh listeners available on each shard of the index. These listeners are used to implement `refresh=wait_for`.

`index.analyze.max_token_count`
: The maximum number of tokens that can be produced using _analyze API. Defaults to `10000`.

`index.highlight.max_analyzed_offset`
: The maximum number of characters that will be analyzed for a highlight request. This setting is only applicable when highlighting is requested on a text that was indexed without offsets or term vectors. Defaults to `1000000`.

`index.max_terms_count`
: The maximum number of terms that can be used in Terms Query. Defaults to `65536`.

`index.max_regex_length`
: The maximum length of regex that can be used in Regexp Query. Defaults to `1000`.

`index.query.default_field`
: (string or array of strings) Wildcard (`*`) patterns matching one or more fields. The following query types search these matching fields by default:

- More like this
- Multi-match
- Query string
- Simple query string

Defaults to `*`, which matches all fields eligible for term-level queries, excluding metadata fields.

`index.routing.allocation.enable`
: Controls shard allocation for this index. It can be set to:

- `all` (default) - Allows shard allocation for all shards.
- `primaries` - Allows shard allocation only for primary shards.
- `new_primaries` - Allows shard allocation only for newly-created primary shards.
- `none` - No shard allocation is allowed.

`index.routing.rebalance.enable`
: Enables shard rebalancing for this index. It can be set to:

- `all` (default) - Allows shard rebalancing for all shards.

- `primaries` - Allows shard rebalancing only for primary shards.
- `replicas` - Allows shard rebalancing only for replica shards.
- `none` - No shard rebalancing is allowed.

`index.gc_deletes`
: The length of time that a deleted document's version number remains available for further versioned operations. Defaults to `60s`.

`index.default_pipeline`
: Default ingest pipeline for the index. Index requests will fail if the default pipeline is set and the pipeline does not exist. The default may be overridden using the `pipeline` parameter. The special pipeline name `_none` indicates no default ingest pipeline will run.

`index.final_pipeline`
: Final ingest pipeline for the index. Indexing requests will fail if the final pipeline is set and the pipeline does not exist. The final pipeline always runs after the request pipeline (if specified) and the default pipeline (if it exists). The special pipeline name `_none` indicates no final ingest pipeline will run.

> **NOTE** You can't use a final pipeline to change the `_index` field. If the pipeline attempts to change the `_index` field, the indexing request will fail.

`index.hidden`
: Indicates whether the index should be hidden by default. Hidden indices are not returned by default when using a wildcard expression. This behavior is controlled per request through the use of the `expand_wildcards` parameter. Possible values are `true` and `false` (default).

Settings in other index modules

edit

Other index settings are available in index modules:

Analysis
: Settings to define analyzers, tokenizers, token filters and character filters.

Index shard allocation
: Control over where, when, and how shards are allocated to nodes.

Mapping
: Enable or disable dynamic mapping for an index.

Merging
: Control over how shards are merged by the background merge process.

Similarities
: Configure custom similarity settings to customize how search results are scored.

Slowlog
: Control over how slow queries and fetch requests are logged.

Store
: Configure the type of filesystem used to access shard data.

Translog
: Control over the transaction log and background flush operations.

History retention
: Control over the retention of a history of operations in the index.

Indexing pressure
: Configure indexing back pressure limits.

X-Pack index settings

edit

Index lifecycle management
: Specify the lifecycle policy and rollover alias for an index.

« Reading indices from older Elasticsearch versions

Analysis »

On this page

Index Settings

- Static index settings
- Dynamic index settings
- Settings in other index modules
- X-Pack index settings

Most Popular

VIDEO
Get Started with Elasticsearch

VIDEO
Intro to Kibana

VIDEO
ELK for Logs & Metrics